

1. Binary message.

message structure: [header, user_data, tail]

header:

header =

[	byte 1		byte 2		byte 3		byte 4	]
[	msg_size LSB		msg_size MSB		flags 8bit		cmdID 8bit	]

message size (**msg_size**) includes size of header, user_data (and tail if integrity flag is 'on').

cmdID=0x00 is special echo command and should not be used.

flags:

[	bit 8		bit 7		bit 6		bit 5	]
[	reserved		reserved		reserved		reserved	]

[	bit 4		bit 3		bit 2		bit 1	]
[	reserved		INTEGRITY_STAMP		SUPPRESS_REPLY_HEADER		EXTENDED_HEADER	]

EXTENDED_HEADER : should be 0 for now.

SUPPRESS_REPLY_HEADER:

if 0:

reply will include header: [header, data]. this is recommended option.

if 1:

no header inserted in reply. reply structure: [data]

INTEGRITY_STAMP:

if 0:

tail will not include integrity stamp.

if 1:

a) tail will include integrity stamp (described in 'tail' section).

b) reply will also include integrity stamp of the same structure in next versions

tail:

If command includes integrity stamp bit then reply will include tail automatically in following format:

tail =

[	byte 1		byte 2		byte 3		byte 4	]
[	fletcher32 LSB byte 0		fletcher32 byte 1		fletcher32 byte 2		fletcher32 MSB byte 3	]

implementation of fletcher32 attached.

2. Reply

reply syntax: [header, data, tail]

header:

header =

[	byte 1		byte 2		byte 3		byte 4	]
[	msg_size LSB		msg_size MSB		flags 8bit		replyStatus	]

message size (**msg_size**) includes size of header and data (and tail if integrity flag is 'on').

flags:

[	bit 8		bit 7		bit 6		bit 5	]
[	reserved		reserved		reserved		reserved	]

[	bit 4		bit 3		bit 2		bit 1	]
[	reserved		INTEGRITY_STAMP		reserved		reserved	]

INTEGRITY_STAMP:

Integrity stamp is added automatically if in incoming message integrity stamp is on.

if 0: tail will not include integrity stamp.

if 1: tail will include integrity stamp (described in 'tail' section).

replyStatus :

0x00 - NO ERROR

0x01 - command not found

0x02 - test of integrity stamp failed (checking fletcher32 checksum of message header and body,
implementation of fletcher32 attached)

0x03 - message length is too long

0x04 - message length is too short

0x05 - not supported flag

0x06 – message size should be even

tail:

If command includes integrity stamp bit then reply will include tail automatically in following format:

tail =

[	byte 1		byte 2		byte 3		byte 4	]
[	fletcher32 LSB byte 0		fletcher32 byte 1		fletcher32 byte 2		fletcher32 MSB byte 3	]

implementation of fletcher32 attached.

a) implementation of fletcher32 for even size of messages:

```
static uint32_t fletcher32(const uint16_t* data, size_t len)
{
    uint32_t sum1 = 0xffff, sum2 = 0xffff;
    size_t tlen;

    while (len) {
        tlen = len > 359 ? 359 : len;

        len -= tlen;

        do {
            sum1 += *data++;
            sum2 += sum1;
        } while (--tlen);

        sum1 = (sum1 & 0xffff) + (sum1 >> 16);
        sum2 = (sum2 & 0xffff) + (sum2 >> 16);
    }
    /* Second reduction step to reduce sums to 16 bits */
    sum1 = (sum1 & 0xffff) + (sum1 >> 16);
    sum2 = (sum2 & 0xffff) + (sum2 >> 16);
    return sum2 << 16 | sum1;
}
```

b) implementation of fletcher32 for any size of messages and for feeding data in chunks:

```
static uint32_t fletcher32_sum1 = 0xffff;
static uint32_t fletcher32_sum2 = 0xffff;
static size_t fletcher32_frame_count = 0;
static size_t fletcher32_len_left = 0;
static uint8_t fletcher32_curr_shift = 0;
static uint8_t fletcher32_curr_sum2_mult = 0;

static void fletcher32_init(size_t msg_len)
{
    fletcher32_sum1 = 0xffff;
    fletcher32_sum2 = 0xffff;
    fletcher32_frame_count = 0;
    fletcher32_len_left = msg_len;
    fletcher32_curr_shift = 0;
    fletcher32_curr_sum2_mult = 0;
}

static void fletcher32_add(const uint8_t* data, size_t len)
{
    if (0 == fletcher32_len_left) return;
    while (len--)
    {
        fletcher32_sum1 += (*data++) << fletcher32_curr_shift;
        fletcher32_curr_shift = 8 - fletcher32_curr_shift;
        fletcher32_sum2 += fletcher32_sum1 * fletcher32_curr_sum2_mult;
        fletcher32_curr_sum2_mult = 1 - fletcher32_curr_sum2_mult;
        fletcher32_frame_count += 1;
        fletcher32_len_left -= 1;

        if (((359 * 2) == fletcher32_frame_count) || (0 == fletcher32_len_left))
        {
            /* in case we have odd length:
            if (fletcher32_curr_sum2_mult) fletcher32_sum2 += fletcher32_sum1;

            fletcher32_frame_count = 0;
            fletcher32_sum1 =
                (fletcher32_sum1 & 0xffff) + (fletcher32_sum1 >> 16);
            fletcher32_sum2 =
                (fletcher32_sum2 & 0xffff) + (fletcher32_sum2 >> 16);
            if (0 == fletcher32_len_left) return;
        }
    }
}
```

```
static uint32_t fletcher32_finish()
{
    /* Second reduction step to reduce sums to 16 bits */
    fletcher32_sum1 = (fletcher32_sum1 & 0xffff) + (fletcher32_sum1 >> 16);
    fletcher32_sum2 = (fletcher32_sum2 & 0xffff) + (fletcher32_sum2 >> 16);
    return fletcher32_sum2 << 16 | fletcher32_sum1;
}
```